

XGBoost, LightGBM ও CatBoost — Universal Parameter Guide (যেকোনো প্রজেক্টে ব্যবহারযোগ্য)

এই নোটটা যেকোনো Machine Learning সমস্যায় (Binary Classification, Multi-class Classification, Regression) ব্যবহার করা যাবে। শুধু `objective` / `loss_function` আর `num_class` পাল্টালেই যেকোনো dataset-এ কাজ করবে।

প্রথমেই বুঝে নিন — কোন সমস্যায় কোন Objective

সমস্যার ধরন	XGBoost <code>objective</code>	LightGBM <code>objective</code>	CatBoost <code>loss_function</code>
Binary Classification	<code>binary:logistic</code>	<code>binary</code>	<code>Logloss</code>
Multi-class Classification	<code>multi:softmax</code> / <code>multi:softprob</code>	<code>multiclass</code>	<code>MultiClass</code>
Regression	<code>reg:squarederror</code>	<code>regression</code>	<code>RMSE</code>
Ranking	<code>rank:pairwise</code>	<code>lambdarank</code>	<code>YetiRank</code>

নিয়ম: Binary বা Regression হলে `num_class` লাগবে না। শুধু Multi-class হলেই `num_class` (XGBoost/LightGBM-এ) দরকার হয়।

1 XGBoost — সম্পূর্ণ Parameter List

```
from xgboost import XGBClassifier # regression হলে XGBRegressor

model = XGBClassifier(
    # ---- সমস্যার ধরন নির্ধারণ ----
```

<code>objective="multi:softmax",</code>	<code># binary:logistic / mul</code>
<code>ti:softmax / reg:squarederror</code>	
<code>num_class=None,</code>	<code># শুধু multi-class হলে</code>
দিন, যেমন <code>num_class=3</code>	
<code>eval_metric="mlogloss",</code>	<code># ট্রেনিং মনিটরিং metric</code>
 <code># ---- Tree সংখ্যা ও গঠন ----</code>	
<code>n_estimators=400,</code>	<code># মোট boosting tree</code>
সংখ্যা	
<code>max_depth=6,</code>	<code># প্রতিটা tree-র সর্বোচ্চ গ</code>
ভীরতা	
<code>min_child_weight=1,</code>	<code># একটা node split হতে</code>
ন্যূনতম sample weight	
<code>gamma=0,</code>	<code># split করার জন্য ন্যূনতম l</code>
oss reduction দরকার	
 <code># ---- Learning Control ----</code>	
<code>learning_rate=0.1,</code>	<code># (alias: eta) প্রতিটা ste</code>
p-এ কতটা শেখা হবে	
 <code># ---- Sampling (Overfitting কমাতে) ----</code>	
<code>subsample=0.8,</code>	<code># প্রতি tree-তে randomly</code>
কত % row ব্যবহার হবে	
<code>colsample_bytree=0.8,</code>	<code># প্রতি tree-তে randomly</code>
কত % feature (column) ব্যবহার হবে	
<code>colsample_bylevel=1.0,</code>	<code># প্রতি level-এ কত % fea</code>
ture ব্যবহার হবে	
<code>colsample_bynode=1.0,</code>	<code># প্রতি split/node-এ কত</code>
% feature ব্যবহার হবে	
 <code># ---- Regularization ----</code>	
<code>reg_alpha=0,</code>	<code># L1 regularization (sp</code>
arsity বাড়ায়)	
<code>reg_lambda=1,</code>	<code># L2 regularization (sm</code>
oothness বাড়ায়)	

```

# ---- Class Imbalance হ্যান্ডলিং (Binary-তে) ----
scale_pos_weight=1,          # positive class-কে কত
টা বেশি গুরুত্ব দেওয়া হবে (imbalanced data)

# ---- Tree Building পদ্ধতি ----
tree_method="auto",          # auto / hist / exact /
approx / gpu_hist
booster="gbtree",            # gbtrees / gblines / d
art
grow_policy="depthwise",      # depthwise / lossguide

# ---- Speed / Engineering ----
n_jobs=-1,                    # কতগুলো CPU core ব্যবহার
হবে
random_state=42,              # Reproducibility-র জন্য
fixed seed
early_stopping_rounds=None,    # কত round performance
না বাড়লে থামবে (fit()-এ eval_set লাগবে)
verbosity=1                    # 0=silent, 1=warning,
2=info, 3=debug
)

```

XGBoost — প্রতিটা প্যারামিটারের ব্যাখ্যা (ক্যাটাগরি অনুযায়ী)

♦ সমস্যা নির্ধারণ

প্যারামিটার	ব্যাখ্যা
<code>objective</code>	কোন ধরনের সমস্যা (classification/regression) এবং কোন loss function ব্যবহার হবে তা ঠিক করে
<code>num_class</code>	Multi-class হলে মোট class সংখ্যা (বাধ্যতামূলক); binary/regression-এ লাগবে না
<code>eval_metric</code>	ট্রেনিং চলাকালীন কোন metric দিয়ে performance track হবে

♦ Tree সংখ্যা ও গঠন

প্যারামিটার	ব্যাখ্যা
<code>n_estimators</code>	মোট কতগুলো boosting tree তৈরি হবে — বেশি হলে ভালো শেখে কিন্তু ট্রেনিং সময় ও overfitting -এর ঝুঁকি বাড়ে
<code>max_depth</code>	প্রতিটা tree কত গভীর পর্যন্ত split হতে পারবে — বেশি গভীরতা মানে জটিল pattern ধরতে পারা, কিন্তু overfitting -এর ঝুঁকি
<code>min_child_weight</code>	একটা node আরও split হওয়ার জন্য ন্যূনতম কত sample/weight দরকার — বড় মান মানে বেশি conservative (কম split)
<code>gamma</code>	Split করার আগে ন্যূনতম কতটা loss কমতে হবে — বড় মান মানে কম, বেশি "worth it" split -ই হবে

◆ Learning Control

প্যারামিটার	ব্যাখ্যা
<code>learning_rate</code>	প্রতিটা tree -র প্রভাব কতটা কমিয়ে যোগ করা হবে — কম মান মানে ধীর কিন্তু স্থিতিশীল শেখা (সাধারণত <code>n_estimators</code> বাড়িয়ে compensate করা হয়)

◆ Sampling (Overfitting নিয়ন্ত্রণ)

প্যারামিটার	ব্যাখ্যা
<code>subsample</code>	প্রতিটা tree তৈরির সময় মোট row -এর কত অংশ randomly নেওয়া হবে
<code>colsample_bytree</code>	প্রতিটা tree তৈরির সময় মোট feature -এর কত অংশ randomly নেওয়া হবে
<code>colsample_bylevel</code>	প্রতিটা tree -র প্রতি level -এ কত % feature নেওয়া হবে
<code>colsample_bynode</code>	প্রতিটা split/node -এ কত % feature নেওয়া হবে (সবচেয়ে সূক্ষ্ম নিয়ন্ত্রণ)

◆ Regularization

প্যারামিটার	ব্যাখ্যা
<code>reg_alpha</code>	L1 regularization — কিছু feature -এর weight একদম শূন্য করে দিতে পারে (feature selection -এর মতো কাজ করে)
<code>reg_lambda</code>	L2 regularization — সব weight -কে ছোট/smooth রাখে, extreme value কমায়

◆ Class Imbalance

প্যারামিটার	ব্যাখ্যা
<code>scale_pos_weight</code>	Binary classification -এ imbalanced dataset হলে positive class -কে বাড়তি গুরুত্ব দিতে ব্যবহার হয় (সাধারণত <code>negative_count/positive_count</code> বসানো হয়)

♦ Tree Building পদ্ধতি

প্যারামিটার	ব্যাখ্যা
<code>tree_method</code>	Tree তৈরির algorithm — <code>hist</code> বড় dataset-এ দ্রুত, <code>exact</code> ছোট dataset-এ নিখুঁত, <code>gpu_hist</code> GPU থাকলে ব্যবহার করুন
<code>booster</code>	<code>gbtree</code> (ডিফল্ট, সবচেয়ে common), <code>gblinear</code> (linear model), <code>dart</code> (dropout সহ tree, extra regularization)
<code>grow_policy</code>	<code>depthwise</code> (সব node সমানভাবে growth) বা <code>lossguide</code> (যেখানে loss বেশি কমে সেখানে আগে grow করে, LightGBM-এর মতো আচরণ)

♦ Speed / Engineering

প্যারামিটার	ব্যাখ্যা
<code>n_jobs</code>	কতগুলো CPU thread ব্যবহার হবে (<code>-1</code> = সবগুলো)
<code>random_state</code>	একই ফলাফল বারবার পাওয়ার জন্য fixed random seed
<code>early_stopping_rounds</code>	Validation score কত round ধরে না বাড়লে ট্রেনিং থামিয়ে দিবে (overfitting রোধ করে, সময়ও বাঁচায়)
<code>verbosity</code>	ট্রেনিং-এর সময় কতটা log/output দেখাবে

2 LightGBM — সম্পূর্ণ Parameter List

```

from lightgbm import LGBMClassifier # regression হলে LGBMRegressor

model = LGBMClassifier(
    # ---- সমস্যার ধরন নির্ধারণ ----
    objective="multiclass",          # binary / multiclass /
    regression                        # শুধু multiclass হলে দিন
    num_class=None,                  # monitoring metric
    metric="multi_logloss",

    # ---- Tree সংখ্যা ও গঠন ----
    n_estimators=400,                # মোট boosting round
    সংখ্যা
    max_depth=-1,                    # tree-র গভীরতা সীমা (-1

```

```

= সীমাহীন)
    num_leaves=31, # প্রতিটা tree-তে সর্বোচ্চ le
af সংখ্যা (সবচেয়ে গুরুত্বপূর্ণ প্যারামিটার)
    min_child_samples=20, # একটা leaf-এ ন্যূনতম sam
ple সংখ্যা

    # ---- Learning Control ----
    learning_rate=0.1, # প্রতিটা step-এ কতটা শেখা
হবে

    # ---- Sampling ----
    subsample=0.8, # (alias: bagging_fract
ion) row sampling
    subsample_freq=1, # কত iteration পরপর ba
gging হবে
    colsample_bytree=0.8, # (alias: feature_fract
ion) column sampling

    # ---- Regularization ----
    reg_alpha=0.0, # L1 regularization
    reg_lambda=0.0, # L2 regularization
    min_split_gain=0.0, # split করতে ন্যূনতম loss
reduction

    # ---- Class Imbalance হ্যান্ডলিং ----
    class_weight=None, # 'balanced' দিলে automa
tic class weight ঠিক করবে
    scale_pos_weight=1, # binary imbalance-এর
জন্য

    # ---- Speed / Engineering ----
    boosting_type="gbdt", # gbdt / dart / goss /
rf
    n_jobs=-1,
    random_state=42,
    max_bin=255, # feature-কে কতগুলো bin

```

-এ ভাগ করা হবে

```
    verbosity=-1                                # -1=silent, 0=warning,  
    1=info  
)
```

LightGBM — প্রতিটা প্যারামিটারের ব্যাখ্যা (ক্যাটাগরি অনুযায়ী)

♦ সমস্যা নির্ধারণ

প্যারামিটার	ব্যাখ্যা
<code>objective</code>	সমস্যার ধরন নির্ধারণ করে (binary/multiclass/regression)
<code>num_class</code>	Multi-class হলে class সংখ্যা (বাধ্যতামূলক)
<code>metric</code>	Monitoring-এর জন্য ব্যবহৃত metric (XGBoost-এর <code>eval_metric</code> -এর সমতুল্য)

♦ Tree সংখ্যা ও গঠন

প্যারামিটার	ব্যাখ্যা
<code>n_estimators</code>	মোট boosting tree সংখ্যা
<code>max_depth</code>	Tree-র গভীরতার সীমা — <code>-1</code> মানে সীমাহীন (num_leaves দিয়েই মূলত নিয়ন্ত্রণ হয়)
<code>num_leaves</code>	সবচেয়ে গুরুত্বপূর্ণ প্যারামিটার — একটা tree-তে সর্বোচ্চ কতগুলো leaf থাকবে; LightGBM leaf-wise growth করে (depth-wise না), তাই এটা মূল complexity নিয়ন্ত্রক
<code>min_child_samples</code>	একটা leaf গঠনের জন্য ন্যূনতম sample সংখ্যা — বড় মান মানে বেশি conservative মডেল

♦ Learning Control

প্যারামিটার	ব্যাখ্যা
<code>learning_rate</code>	প্রতিটা tree-র contribution কতটা কমানো হবে

♦ Sampling

প্যারামিটার	ব্যাখ্যা
<code>subsample</code>	প্রতি iteration-এ randomly কত % row নেওয়া হবে (bagging)
<code>subsample_freq</code>	কত iteration পরপর নতুন করে bagging sample নেওয়া হবে (0 করলে bagging বন্ধ)

প্যারামিটার	ব্যাখ্যা
<code>colsample_bytree</code>	প্রতি tree-তে randomly কত % feature নেওয়া হবে

◆ Regularization

প্যারামিটার	ব্যাখ্যা
<code>reg_alpha</code>	L1 regularization
<code>reg_lambda</code>	L2 regularization
<code>min_split_gain</code>	Split করার জন্য ন্যূনতম কতটা loss কমতে হবে (XGBoost-এর <code>gamma</code> -এর সমতুল্য)

◆ Class Imbalance

প্যারামিটার	ব্যাখ্যা
<code>class_weight</code>	'balanced' দিলে LightGBM automatic ভাবে ছোট class-গুলোকে বেশি গুরুত্ব দিবে
<code>scale_pos_weight</code>	Binary classification imbalance-এর জন্য (class_weight-এর বদলে ব্যবহার করা যায়)

◆ Speed / Engineering

প্যারামিটার	ব্যাখ্যা
<code>boosting_type</code>	<code>gbdt</code> (ডিফল্ট, standard boosting), <code>dart</code> (dropout সহ, বেশি regularized), <code>goss</code> (বড় gradient-ওয়ালা sample-কে priority দেয়, দ্রুত), <code>rf</code> (random forest মোড)
<code>n_jobs</code>	CPU thread সংখ্যা
<code>random_state</code>	Reproducibility seed
<code>max_bin</code>	Continuous feature-কে কতগুলো bin-এ ভাগ করা হবে — বেশি bin মানে বেশি নিখুঁত কিন্তু ধীর
<code>verbosity</code>	Log output-এর পরিমাণ

3 CatBoost — সম্পূর্ণ Parameter List

```
from catboost import CatBoostClassifier # regression হলে Ca
tBoostRegressor
```



```

model = CatBoostClassifier(
    # ---- সমস্যার ধরন নির্ধারণ ----
    loss_function="MultiClass",           # Logloss / MultiClass
    / RMSE
    eval_metric="MultiClass",             # monitoring metric

    # ---- Tree সংখ্যা ও গঠন ----
    iterations=400,                        # মোট boosting round
    সংখ্যা
    depth=6,                              # tree-র গভীরতা (সাধারণত
    4-10)

    # ---- Learning Control ----
    learning_rate=0.1,                    # প্রতিটা step-এ কতটা শেখা
    হবে

    # ---- Regularization ----
    l2_leaf_reg=3.0,                      # L2 regularization (le
    af value smoothing)
    random_strength=1.0,                  # split selection-এ ran
    domness যোগ করে (overfitting কমায়ে)

    # ---- Sampling ----
    bagging_temperature=1.0,              # Bayesian bootstrap-এর
    randomness নিয়ন্ত্রণ করে
    subsample=0.8,                        # (Bernoulli bootstrap
    ব্যবহার করলে) row sampling ratio

    # ---- Categorical Feature হ্যান্ডলিং (CatBoost-এর বিশেষত্ব) ---
    -
    cat_features=None,                    # কোন কোন column categ
    orical তা index/নাম দিয়ে বলে দিতে হয়
    one_hot_max_size=2,                   # কতগুলো unique value প
    র্যন্ত one-hot encoding হবে, তার বেশি হলে অন্য পদ্ধতি

    # ---- Class Imbalance হ্যান্ডলিং ----

```

```

class_weights=None,          # প্রতিটা class-এর জন্য আ
লাদা weight (dict/list আকারে)
auto_class_weights=None,    # 'Balanced' বা 'SqrtBal
anced' দিলে automatic

# ---- Precision / Speed ----
border_count=254,          # Numerical feature কত
গুলো bin-এ ভাগ হবে
thread_count=-1,          # CPU thread সংখ্যা (-1 =
সবগুলো)
random_state=42,
early_stopping_rounds=None, # early stopping (eval_
set লাগবে fit()-এ)
verbose=0                  # 0=silent, নাহলে প্রতি N
iteration পরপর progress দেখাবে
)

```

CatBoost — প্রতিটা প্যারামিটারের ব্যাখ্যা (ক্যাটাগরি অনুযায়ী)

♦ সমস্যা নির্ধারণ

প্যারামিটার	ব্যাখ্যা
<code>loss_function</code>	কোন loss optimize হবে তা ঠিক করে (Logloss/MultiClass/RMSE ইত্যাদি)
<code>eval_metric</code>	Performance monitor করার metric

♦ Tree সংখ্যা ও গঠন

প্যারামিটার	ব্যাখ্যা
<code>iterations</code>	মোট boosting round/tree সংখ্যা (<code>n_estimators</code> -এর সমতুল্য)
<code>depth</code>	প্রতিটা tree-র গভীরতা — CatBoost symmetric tree ব্যবহার করে তাই সাধারণত depth কম (4-10) রাখা হয়

♦ Learning Control

প্যারামিটার	ব্যাখ্যা
<code>learning_rate</code>	প্রতিটা step-এ model কতটা আপডেট হবে

◆ Regularization

প্যারামিটার	ব্যাখ্যা
<code>l2_leaf_reg</code>	L2 regularization — leaf value-কে বেশি extreme হতে বাধা দেয়
<code>random_strength</code>	Split score-এ randomness যোগ করে, যাতে মডেল সবসময় একই রকম "সবচেয়ে ভালো" split না বেছে overfitting এড়াতে পারে

◆ Sampling

প্যারামিটার	ব্যাখ্যা
<code>bagging_temperature</code>	Bayesian bootstrap-এ কতটা randomness/বৈচিত্র্য থাকবে — বেশি মান মানে বেশি এলোমেলো sample weight
<code>subsample</code>	(Bernoulli bootstrap mode-এ) প্রতিটা tree-তে কত % row ব্যবহার হবে

◆ Categorical Feature হ্যান্ডলিং (CatBoost-এর সবচেয়ে বড় বিশেষত্ব)

প্যারামিটার	ব্যাখ্যা
<code>cat_features</code>	কোন column-গুলো categorical (যেমন: "city", "gender") তার index বা নাম — CatBoost নিজে থেকেই categorical encoding করে ফেলে, manual one-hot/label encoding লাগে না
<code>one_hot_max_size</code>	কোনো categorical feature-এ unique value সংখ্যা এই মানের নিচে থাকলে one-hot encoding হবে, বেশি হলে CatBoost-এর নিজস্ব target-encoding পদ্ধতি ব্যবহার হবে

◆ Class Imbalance

প্যারামিটার	ব্যাখ্যা
<code>class_weights</code>	প্রতিটা class-এর জন্য আলাদা করে weight বসানো যায় (dict/list আকারে, যেমন <code>{0:1, 1:3}</code>)
<code>auto_class_weights</code>	'Balanced' দিলে automatic ভাবে sample সংখ্যার উল্টো অনুপাতে weight বসাবে

◆ Precision / Speed

প্যারামিটার	ব্যাখ্যা
<code>border_count</code>	Numerical feature কতগুলো bin-এ ভাগ হবে (XGBoost-এর <code>max_bin</code> -এর সমতুল্য)
<code>thread_count</code>	CPU thread সংখ্যা

প্যারামিটার	ব্যাখ্যা
<code>random_state</code>	Reproducibility seed
<code>early_stopping_rounds</code>	Validation score না বাড়লে কত round পর থামবে
<code>verbose</code>	Progress log দেখানোর পরিমাণ

তিনটা লাইব্রেরির Parameter Naming তুলনা (Universal Cheat Sheet)

ধারণা	XGBoost	LightGBM	CatBoost
Tree/Round সংখ্যা	<code>n_estimators</code>	<code>n_estimators</code>	<code>iterations</code>
Tree-র গভীরতা	<code>max_depth</code>	<code>max_depth</code> / <code>num_leaves</code>	<code>depth</code>
Learning step	<code>learning_rate</code>	<code>learning_rate</code>	<code>learning_rate</code>
L1 Regularization	<code>reg_alpha</code>	<code>reg_alpha</code>	— (নেই)
L2 Regularization	<code>reg_lambda</code>	<code>reg_lambda</code>	<code>l2_leaf_reg</code>
Row Sampling	<code>subsample</code>	<code>subsample</code>	<code>subsample</code> / <code>bagging_temperature</code>
Column Sampling	<code>colsample_bytree</code>	<code>colsample_bytree</code>	(automatic)
Min split gain	<code>gamma</code>	<code>min_split_gain</code>	(<code>random_strength</code> দিয়ে similar effect)
Class Imbalance	<code>scale_pos_weight</code>	<code>class_weight</code> / <code>scale_pos_weight</code>	<code>class_weights</code> / <code>auto_class_weights</code>
Multi-class label সংখ্যা	<code>num_class</code> (আবশ্যিক)	<code>num_class</code> (আবশ্যিক)	প্রয়োজন নেই (automatic)
Categorical feature handling	manual encoding লাগে	manual/native (categorical_feature প্যারামিটার আছে)	native, <code>cat_features</code> দিলেই যথেষ্ট
CPU thread	<code>n_jobs</code>	<code>n_jobs</code>	<code>thread_count</code>
Bin সংখ্যা	(<code>tree_method='hist'</code> এ ব্যবহৃত)	<code>max_bin</code>	<code>border_count</code>
Random seed	<code>random_state</code>	<code>random_state</code>	<code>random_state</code>

🔑 eval_metric / metric আর্গুমেন্ট লিস্ট

XGBoost

rmse, mae, logloss, mlogloss, error, merror, auc, aucpr, map, ndcg

LightGBM

multi_logloss, multi_error, binary_logloss, auc, auc_mu, rmse, mae, l1, l2

CatBoost

Logloss, MultiClass, MultiClassOneVsAll, Accuracy, TotalF1, AUC, Precision, Recall, Kappa, wKappa, RMSE, MAE



যেকোনো নতুন প্রজেক্টে ব্যবহার করার সময় মনে রাখুন

1. সমস্যার ধরন প্রথমে ঠিক করুন — binary/multiclass/regression অনুযায়ী `objective` / `loss_function` বদলান।
2. **Multi-class** হলে `num_class` ভুলবেন না (XGBoost, LightGBM-এ) — CatBoost-এ লাগবে না।
3. **Class imbalance** থাকলে — `scale_pos_weight` (binary), `class_weight='balanced'` (LightGBM), বা `auto_class_weights='Balanced'` (CatBoost) ব্যবহার করুন।
4. **Categorical feature** থাকলে — CatBoost সবচেয়ে সুবিধাজনক (`cat_features` দিলেই হয়, manual encoding লাগে না)।
5. **GridSearchCV**-তে শুধু tune করতে চাওয়া প্যারামিটারগুলোই `param_grid` এ দিন; বাকি fixed সেটিং (`objective`, `random_state` ইত্যাদি) constructor-এ দিন।
6. `n_jobs=1` GridSearchCV-তে রাখুন যখন model নিজে multi-threaded (XGBoost/LightGBM/CatBoost) — নাহলে CPU oversubscription হতে পারে।
7. **Overfitting** মনে হলে — `max_depth` / `depth` কমান, `subsample` / `colsample_bytree` কমান, বা `reg_alpha` / `reg_lambda` / `l2_leaf_reg` বাড়ান।
8. **Underfitting** মনে হলে — `n_estimators` / `iterations` বাড়ান, `learning_rate` কমিয়ে `n_estimators` বাড়ান, বা `max_depth` / `num_leaves` বাড়ান।